



로그 재현 기록

김건우 | 김명진 | 김성현 | 이진우



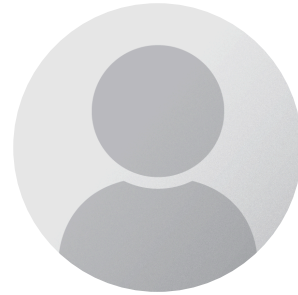
김건우



김명진



김성현



이진우

Cassandra

Step to reproduce

```
cd ./cassandra
direnv allow # load old python and java...
wget https://archive.apache.org/dist/cassandra/2.1.1/apache-cassandra-2.1.1-bin.tar.gz
tar -xvf apache-cassandra-2.1.1-bin.tar.gz
sed -i 's/<root level="INFO"/><root level="DEBUG"/>/g' ./apache-cassandra-2.1.1/conf/
logback.xml
./apache-cassandra-2.1.1/bin/cassandra -f # in other shell!
./apache-cassandra-2.1.1/bin/cqlsh -f prepare.cql
cd ../triger
direnv allow
python CASSANDRA-8280.py
# ctrl+c
cd ..
cp apache-cassandra-2.1.1/logs/system.log log
```

CASSANDRA-8280

카산드라 서버에 exception을 발생시키는 이슈입니다.

Step to reproduce

```
direnv allow # load old python and java...
wget https://archive.apache.org/dist/cassandra/2.1.2/apache-cassandra-2.1.2-bin.tar.gz
tar -xvf apache-cassandra-2.1.2-bin.tar.gz
sed -i 's/<root level="INFO"/><root level="DEBUG"/>/g' ./apache-cassandra-2.1.2/conf/
logback.xml
./apache-cassandra-2.1.2/bin/cassandra -f # in other shell!
./apache-cassandra-2.1.2/bin/cqlsh -f stress.cql --debug 2>&1 | tee cqlsh-log
# cassandra 서버에서 버그가 일어나지는 않지만 cqlsh와 통신과정에서 많은 로그가 생성되기 때문에 기록
cp ./apache-cassandra-2.1.2/logs/system.log cassandra-log
```

CASSANDRA-8351

이 이슈는 cassandra 서버 자체의 버그가 아닌, 부속 프로그램인 cqlsh에 일어나는 segfault에 관한 내용입니다.

화면 마지막에 출력되는 segfault 라인은 실행 중이던 프로세스에서 나온 로그가 아닌 실행할 때 사용한 shell의 메시지입니다.

bash의 경우:

```
Segmentation fault (core dumped)
```

fish의 경우:

```
fish: Job 1, './apache-cassandra-2.1.2/bin/cq...' terminated by signal SIGSEGV (Address  
boundary error)
```

이 메시지는 리다이렉션 되지 않기 때문에 남겨진 로그 파일에는 남지 않습니다.

Step to reproduce

```
direnv allow # load old python and java...  
wget https://archive.apache.org/dist/cassandra/3.9/apache-cassandra-3.9-bin.tar.gz  
tar -xvf apache-cassandra-3.9-bin.tar.gz  
./apache-cassandra-3.9/bin/cassandra -f # in other shell!  
./apache-cassandra-3.9/bin/cqlsh -f trigger.cql
```

restart cassandra server then you will get the error.

```
cp ./apache-cassandra-3.9/logs/debug.log ./log
```

CASSANDRA-13669

카산드라 서버에 특정 쿼리를 진행하면 서버 재시작시에 크래시가 일어나는 이슈입니다.

Step to reproduce

```
cd ./cassandra
sudo docker compose up -d
cd ../trigger
direnv allow
mvn clean package
cd ../cassandra
sudo docker compose down
cp ./log/debug.log log
```

CASSANDRA-15896

카산드라 서버에 exception을 발생시키는 이슈입니다.

Mongodb

SERVER-51733

1 → 2 → 3

<https://jira.mongodb.org/browse/SERVER-51733>

1. 컨테이너 실행

```
sudo docker compose up -d
```

2. 이후 MongoDB shell에서 진행필요

```
sudo docker exec -it mongo51733 \
mongo localhost:27019 \
--authenticationDatabase "admin" \
-u "root" \
-p "DontTryThisIsatHome"
```

3. config서버 인증을 활성화 -> 크래시발생!

```
rs.initiate({  
  _id:"configs",  
  configsvr:true,  
  members:[  
    {  
      _id:0,  
      host:"localhost:27019"  
    }  
  ]  
})
```

SERVER-77168

MongoDB Sharded Cluster 환경. Config Server 초기화 과정 중 Replica Set 설정, Config 데이터베이스 초기화 과정에서 발생!

재현 방법

```
mkdir ./data
cp ./repro.sh ./data
chmod 777 ./data
chmod 777 ./data/repro.sh
docker compose up
sudo cp ./data/mongod.log log
sudo chown $(id -u) log
```

SERVER-101180

1 → 2 → 3

<https://jira.mongodb.org/browse/SERVER-101180>

1. 컨테이너 실행

```
sudo docker compose up -d
```

2. MongoDB Shell 접속

```
sudo docker exec -it mongo101180 mongosh
```

3. 재현 코드 실행

MongoDB Shell에서 아래 코드를 그대로 실행한다.

```
kCollName = "boom";  
db[kCollName].insert({_id: "X".repeat(16776704)});  
db[kCollName].remove({});
```

간단한 설명!

MongoDB의 BatchedDelete 처리 과정에서 invariant failure가 발생해서 mongoDB가 비정상 종료된다.

SERVER-115200

1 → 2 → 3 → 4 → 5

<https://jira.mongodb.org/browse/SERVER-115200>

1. 컨테이너 실행

```
sudo docker compose up -d
```

2. 더미데이터 삽입 (중요 x, 저널 생성을 위해서 진행)

```
cat > ~/repro115200_insert.js <<'JS'
const dbx = db.getSiblingDB("repro115200");

dbx.t.drop();

for (let b = 0; b < 30; b++) {
  const docs = [];
```

```
for (let i = 0; i < 1000; i++) {  
  docs.push({  
    _id: b * 1000 + i,  
    pad: "x".repeat(2000),  
    ts: new Date()  
  });  
}  
  
dbx.runCommand({  
  insert: "t",  
  documents: docs,  
  writeConcern: { j: true }  
});  
  
print("batch", b, "inserted");  
}  
  
print("count:", dbx.t.countDocuments());  
JS
```


위 파일을 컨테이너에 복사하고, 실행한다.

```
sudo docker cp ~/repro115200_insert.js \
mongo115200:/tmp/repro115200_insert.js
```

```
sudo docker exec mongo115200 \
mongosh --quiet /tmp/repro115200_insert.js
```

3. mongod 종료

```
sudo docker kill -s SIGKILL mongo115200
```

4. WiredTiger journal 손상시키기!!

journal 파일 일부

```
0xffffffff
```

로 덮어써서 손상 만들기.

```
sudo python3 - "$WTLOG" <<'PY'  
import sys  
  
p = sys.argv[1]  
offset = 128  
  
with open(p, "r+b") as f:  
    f.seek(offset)  
    before = f.read(16)  
  
    f.seek(offset)  
    f.write(b"\xff\xff\xff\xff")  
  
print("patched:", p)  
print("offset:", offset)  
print("before:", before.hex())  
print("after: ffffffff")  
PY
```

5. MongoDB 재시작

```
cd ~/mongo115200  
sudo docker compose start
```

Zookeeper

zk2247

ZOOKEEPER-2247

관련 자료

- <https://issues.apache.org/jira/browse/ZOOKEEPER-2247> 이슈원본
- <https://zookeeper.apache.org/doc/r3.5.0-alpha/> 공식문서
- <https://archive.apache.org/dist/zookeeper/> 소스코드

-   Mohammad Arshad added a comment - 19/Aug/15 12:11
In test environment, after the zookeeper server start, disk is made read-only using an internal tool which executes commands similar to `mount -o ro,remount /dev/sda3`. yes, it is not a disk full scenario

기반 환경 세팅

1. opentofu (+ terraform-provider-incus) + cloud-init 을 통한 VM 생성

- 0_CLOUDS/iac/incus-infra/main.tf
- 0_CLOUDS/iac/incus-infra/cloudinit.yaml
- (우분투 환경임)

2. 도커 컨테이너

- how to install docker in ubuntu 문서 참고

핵심 환경 세팅

- Dockerfile (여기에 아예 Apache archive 경로에서 curl 하게 했음) -> base image 는 이클립스 테무린-8버전 (curl,tar 있음)
 - docker-compose.yml (3노드 쉽게 띄우는 용도)
 - entrypoint.sh (myid 생성 및 진입점 start)
 - zoo.cfg (주키퍼 설정)
 - log4j.properties (특정 주키퍼 버전 이하에서만) (로그 설정 :)
 - logback.xml (특정 주키퍼 버전 이후)
1. Dockerfile에서 원하는 버전의 zk 적기
 2. docker build -t zk버전:local . //점 필수임!!!!!!!!!!!!!!
 3. docker images 확인
 4. docker-compose.yml의 images 버전을 위 빌드한 이미지로 맞추기
 5. /mnt/zk-txn 만들기 //이거 뭐 만드는 방법이 있음...
 6. 마운트 하기
 7. docker-compose.yml에 한 노드만 /mnt/zk-txn에 바인딩 하기
 8. docker compose up -d -force-recreate

9. docker ps 로 상태 확인
10. docker compose exec zk1~3 bin/zkServer.sh status //팔로워,리더 됨. 이때 /mnt마운트 된 노드가 리더가 되게 하기
11.

```
echo "create /test hello" | docker compose exec -T zk1 bin/zkCli.sh -server zk3:2181
```

 - 트랜잭션 로그를 생성하는 명령. -T는 tty끄기 옵션
 - 트랜잭션 로그할 때, 각 이름이? 달라야한다고 함 여러번 보낼 때 주의
12. docker compose down
13. sudo mount -o remount,bind,ro /mnt/zk-txn 쓰기 불가 상태
14. docker compose up -d
15.

```
echo "create /after "boom" | docker compose exec -T zk1 bin/zkCli.sh -server zk3:2181
```
16. [예상] 트랜잭션 로그 쓰기 실패
17. sudo mount -o remount,bind,rw /mnt/zk-txn 쓰기 가능으로 다시 바꾸기
18. [예상] docker ps 3 node 정상

19. [예상] `docker compose exec zk1~3 bin/zkServer.sh status` 여전히 리더가 바뀌지 않음

20. [예상]

```
echo "create /validate "hello2" | docker compose exec -T zk1 bin/zkCli.sh -server zk3:2181
```

여전히 쓰기 실패

21. [예상] 트랜잭션 로그 쓰기는 안되면서 리더에서 내려오지는 않는 에러가 발생

ZOOKEEPER-2592

zookeeper 실행 상태에서 디스크 용량이 가득 찼을 때 발생한 응답 불능이 디스크 용량이 다시 확보된 상황에서도 지속되는 버그

디스크 용량이 가득 찬 상황을 구현하기 위해 qemu vm을 사용함

로그

Redis

Overview

AOF 재작성 등을 위해 백그라운드 프로세스를 `fork()`하다가 실패할 경우 부모-자식 간 통신을 위해 미리 열어둔 파이프 리소스가 닫히지 않고 그대로 누수되어 한계에 도달하게되면 새로운 I/O 핸들을 요구하는 모든 시스템 콜(`accept`, `pipe`, `open`, `socket`)이 `EMFILE(Too many open files)` error를 반환하며 차단됨

Step to Reproduce

```
sudo chmod 777 log
```

```
sudo chmod +x ./trigger.sh
```

```
sudo docker compose up -d -build
```

```
./trigger.sh
```

“누수된 FD 개수가 더이상 증가하지 않을 때 CTRL + C 강제종료”

```
sudo docker compose down
```

Check

`./log/baseline.log`에 반복적인 `fork()`실패 후 `Too many open files error` 확인

Overview

방대한 쓰기요청으로 인한 메모리 파편화 발생 시 기존 키를 방출하여 확보하는 메모리보다 새로 들어오는 메모리 속도가 빠르면 논리적으로 maxmemory 제한을 유지하더라도 실제 물리 메모리 사용량은 제한을 초과하여 OOM killer가 강제종료

Step to reproduce

```
sudo chmod 777 log
```

```
docker compose up -d
```

```
./fragtrigger.sh
```

```
docker compose down
```

Check

[system log] ./log/baseline.log 에서 virtual memory 104854824 사용중이라는 로 그 확인

```
“ - 60 clients connected (0 slaves), 104854824 bytes in use”
```

[kernel log] sudo dmesg -T | tail -20 으로 메모리 초과로 인한 OOM Killer 개입
확인

```
“oom-kill:constraint=CONSTRAINT_MEMCG,nodemask=(null),cpuset=docker-  
adba9ae041f921f696bfff13a02993c62c53f2eb95f4a9e665ea578c98d4262c.scope,mems_allc  
system.slice/docker-  
adba9ae041f921f696bfff13a02993c62c53f2eb95f4a9e665ea578c98d4262c.scope,task_memo  
system.slice/docker-  
adba9ae041f921f696bfff13a02993c62c53f2eb95f4a9e665ea578c98d4262c.scope,task=red  
server,pid=107327,uid=999”
```

```
“Memory cgroup out of memory: Killed process 107327 (redis-server) total-  
vm:209580kB, anon-rss:133060kB, file-rss:3072kB, shmem-rss:0kB, UID:999  
pgtables:416kB oom_score_adj:0”
```

Overview

- 다중 프로세스가 대량의 더미데이터(dd)를 생성하고 전역 동기화(sync)를 반복호출하여 OS 커널의 VFS(virtual file system) Lock을 장기간 점유
- 이 때 매초 발생하도록 설정한 appendfsync 또한 대기
- 동시에 벤치마크 클라이언트들이 redis 메인 스레드로 지속적인 SET명령어(쓰기요청) 전송
- 메인 스레드는 SET명령어를 AOF 버퍼에 쓰기 직전 fsync 지연 상태 확인. 2초 초과 감지 시 OOM을 막기위해 blocking 후 디스크에 경고 로그 출력(방어로직)
- 메인 스레드가 blcok상태가 되어 어떠한 커맨드 응답도 반환받지 못하므로 네트워크 큐에서 대기하던 클라이언트 애플리케이션에서 timeout exception 발생

Step to reproduce

```
mkdir data
```

```
sudo chmod 777 data log
```

```
sudo chmod +x trigger/*.sh
```

```
sudo docker compose up -d
```


redis-13099

```
cd trigger
```

```
./01-stress-io.sh && ./02-run-benchmark.sh
```

```
cd ..
```

```
sudo docker compose down
```

Check

./log/baseline.log에서 경고 로그 확인

“ * Asynchronous AOF fsync is taking too long (disk is busy?). Writing the AOF buffer without waiting for fsync to complete, this may slow down Redis. “

Requirements

- 실행하는 유저의 그룹에 docker를 추가할 것(docker 명령을 sudo 없이 사용 가능)
- python 설치하기 혹은 direnv allow

Step to reproduce

```
mkdir ./data  
chmod 777 ./data  
chmod 666 ./data/*  
  
./run.sh
```

check

```
./log ./monitor-log
```

redis-15424

특정 동작 수행 중 확률적으로 crash가 발생하는 이슈

keys가 5000이 넘어가도 crash가 발생하지 않으면 재시도 추천

마지막 페이지